Lecture 16: Introduction to Complexity Theory

Reference: Chapters 1–2, Arora & Barak

Overview

Topics covered:

- Decision Problems
- Turing Machines
- Uncomputable Functions
- Halting Problem

Complexity Theory asks: *What can we compute with limited resources (time, memory, ...)?*

There are two sides to this question:

- **Algorithms side:** What we *can* compute safely in a given amount of time.
- **Lower bounds side:** Proving *limits* on what we can do — showing we *cannot* solve certain problems with some given amount of resources.

We need mathematics to reason about this. The goal is a **mathematical model** of computational problems and algorithms.

Decision Problems

What Kind of Computational Problems?

We focus on **decision problems**: problems with **yes/no answers**.

We don't lose much generality by doing so.

Example 2.1 (Independent Sets in Graphs)**.**

- **Decision problem** $\mathrm{Ind}(G, k)$: Does graph G have an independent set of size k?
- **Optimization problem**: Given G, calculate the maximum independent set size.

Reduction: For $k = 0, 1, \ldots, n$, solve $\mathrm{Ind}(G, k)$. The maximum independent set size is the *last* k for which the answer is **yes**. So solving the decision problem also solves the optimization problem.

Example 2.2 (Search Problem)**.** Given G and k, *find* an independent set I of size k.

Order vertices $v_1, \ldots, v_n$. Ask:

1. Does v_1 belong to an independent set of size k?
   - **Yes** $\Rightarrow$ Is v_2 in an independent set of size $k-1$ in $G \setminus (\{v_1\} \cup N(v_1))$? i.e., solve $\mathrm{Ind}(G \setminus (\{v_1\} \cup N(v_1)),\ k-1)$.

- **No** $\Rightarrow$ Is v_2 in an independent set of size k in $G \setminus \{v_1\}$?

2. Continue recursively until the full independent set is constructed.

Solving the decision problem suffices to solve the search problem.

Mathematical Definition of a Decision Problem

A decision problem is mathematically a **set of strings** (a *language*).

- $x \in \{0,1\}^n$ is a binary string of length n.
- $\{0,1\}^* = \bigcup_{n=0}^{\infty} \{0,1\}^n$ denotes *all finite binary strings.*

Definition 2.1 (Boolean Function on Strings)**.** A **boolean function on strings** is a function

$$f : \{0,1\}^* \to \{0,1\}.$$

The **language defined by** f is

$$L_f = \{x \in \{0,1\}^* : f(x) = 1\} \subseteq \{0,1\}^*.$$

Definition 2.2 (Decision Problem)**.** Given $x \in \{0,1\}^*$, compute $f(x)$, or equivalently, determine whether $x \in L_f$.

Decision problems are thus **languages** (subsets of strings) or **boolean functions** on strings.
Remark. This is a very general model: we can encode graphs, weighted graphs, arrays of numbers, families of subsets, etc. as strings. The function f defines a yes/no question about its input.

Turing Machines

Motivation

How do we model an algorithm or computer mathematically? What does it mean for an algorithm to *compute* a function f or *recognize* a language L?

Definition

A **Turing Machine (TM)** has a set of *tapes*, each with a **read/write head** that can read and write symbols and move along the tape.

- **Tape 1**: Input tape (read-only).
- **Tapes 2, . . . ,** $k-1$: Work tapes.
- **Tape** k: Output tape (write output $f(x)$ here).

At each step, we read the k symbols from current head locations, possibly write symbols in tapes $2, \ldots, k$ (leaving input tape unchanged), and possibly move some or all heads.
Definition 3.1 (Turing Machine (Formal))**.** A Turing Machine M consists of:

- A finite **alphabet** $\Gamma = \{\sqcup, 0, 1, \ldots, \gamma\}$ (includes a blank symbol $\sqcup$).

- A finite set of **states** $Q = \{Q_{\text{init}}, Q_{\text{final}}, Q_1, Q_2, \ldots, Q_r\}$.
- A **transition function**

$$\delta : Q \times \Gamma^k \;\rightarrow\; Q \times \Gamma^{k-1} \times \{L, R, S\}^k$$

(where L = move left, R = move right, S = stay).

Idea: The TM follows the rules defined by δ and tries to end in state Q_{final} with $f(x)$ written on the output tape.

Definition 3.2 (TM Computes a Function)**.** A Turing Machine M **computes** a function $f : \{0,1\}^* \rightarrow \{0,1\}$ if, when initiated with x on the input tape, the machine has $f(x)$ written on the output tape when it reaches state Q_{final} in a finite number of steps (that may depend on x).

Example 3.1. Let $f(x) = 1$ if x contains at least one `1`, and 0 otherwise.

Turing Machine:

$$\Gamma = \{0, 1, \sqcup\}, \quad Q = \{Q_{\text{init}},\ Q_{\text{no}},\ Q_{\text{final}}\}$$

Transition function δ (two tapes: input, output):

1. On Q_{init}: transition to Q_{no}, write `0` to output tape.
2. On Q_{no}:
   - If input tape reads `0`: stay in Q_{no}, move input head right (R).
   - If input tape reads $\sqcup$ (blank): transition to Q_{final}.
   - If input tape reads `1`: write `1` to output tape, transition to Q_{final}.

This TM runs in $(n+1)$-time (at most $n+1$ steps for input of length n).

Time Complexity

Definition 3.3 (TM Computes in $T(n)$-Time)**.** A Turing Machine M **computes a function** f **in** $T(n)$**-time** if M computes $f(x)$ on input x in at most $T(|x|)$ steps, where $|x|$ denotes the length of string x.

Here $T : \mathbb{N} \rightarrow \mathbb{N}$.

Remark. The precise details of the machine (alphabet size, number of tapes, whether you can write on the input tape, ...) don't matter much. If we can compute f in one model in time $T(n)$, we can compute it in another model in "comparable" time, e.g., $T(n) \cdot \log |T|$, $T(n)^2$, ... These are all equivalent for **polynomial-time** computations.

Uncomputable Functions

Can All Functions Be Computed?

No! There are *uncomputable* functions.

Idea. We can **encode** any Turing Machine M as a finite string, written $\langle M \rangle$. There is a **Universal Turing Machine** U that takes $(\langle M \rangle, x)$ as input and outputs $M(x)$ — essentially acting as a *compiler/interpreter*.

It is useful to imagine giving $\langle M \rangle$ to M as an input: feeding source code into an algorithm as its own input (weird thing to do!).

Theorem 4.1. *There exist uncomputable functions.*

Proof. Define the function $\mathrm{UC} : \{0,1\}^* \to \{0,1\}$ as follows. On input x:

1. Run Turing Machine M where $\langle M \rangle = x$, on input x.
2. If M stops in finite time and outputs `1`: $\mathrm{UC}(x) = 0$.
3. Otherwise (M doesn't stop, or outputs `0`): $\mathrm{UC}(x) = 1$.

Claim: No Turing Machine can compute UC.

Assume for the sake of contradiction there is a TM M that computes UC. What is $\mathrm{UC}(\langle M \rangle)$?

By definition of UC:

$$\mathrm{UC}(\langle M \rangle) = \begin{cases} 0 & \text{if } M(\langle M \rangle) \text{ halts in finite time and outputs } 1 \\ 1 & \text{otherwise} \end{cases}$$

Since M computes UC, we have $M(\langle M \rangle) = \mathrm{UC}(\langle M \rangle)$. But then:

$$M(\langle M \rangle) = 1 \iff \mathrm{UC}(\langle M \rangle) = 0 \iff M(\langle M \rangle) = 0.$$

This is a **contradiction**. Therefore no such M can exist. □

(This is a "diagonalization" argument.) □

Remark. The function UC may seem contrived. The next result shows a very practical uncomputable problem.

The Halting Problem

Practical question: Does my code have an infinite loop?

Definition 5.1 (Halting Problem)**.** On input $(\langle M \rangle, x)$, does Turing Machine M halt in finite time on input x?

Theorem 5.1. *The Halting Problem is* ***not computable****.*

Proof. Suppose for the sake of contradiction there is a TM H that computes the Halting Problem. We will show that using H we can compute UC, but this is impossible — contradiction. This is a **proof by reduction**.

Define a TM M_{UC} that does the following on input x:

1. Run H on input (x, x).

2. If H outputs **no** (i.e., M does not halt on x): output `1`.
3. If H outputs **yes** (i.e., M halts on x): use Universal TM U to run $\langle x \rangle$ on x.
   - If this outputs `1`: output `0`.
   - If this outputs `0`: output `1`.

This machine M_{UC} computes UC — but we proved UC is uncomputable. Contradiction. Therefore, no TM H solving the Halting Problem can exist. □

(Proof by reduction.) □